PROTECTED TEMPORAL HIERARCHY GROWTH WITH VERSIONED COMPOSITIONAL MEMORY

Technical field

[0001] Event-driven learning systems that grow temporal representation hierarchies while preserving an acquired processing path.

Background

[0002] Adding stages can change an acquired actor through feedback state, random-stream use or optimizer ownership even when an added output is initialized to zero. Coarse role slots can also merge distinct lower memories needed for composition.

Summary

[0003] A growth controller preserves an acquired actor path and gives added temporal computation private ownership. A hierarchical embodiment binds parent versions to identified lower-memory versions, with roles separate from child identity.

Brief description of the drawings

[0004] FIG. 1 is a block diagram of an implementation. FIG. 2 shows operations of the implementation. Reference signs identify the same functional elements throughout the drawings.

Detailed description

[0005] In FIG. 1, 100 identifies events; 110 identifies parent path; 120 identifies private branch; 130 identifies bindings; 140 identifies output; 150 identifies learning. In FIG. 2, 210 identifies child version, 220 identifies parent binding and 230 identifies later query.

[0006] A protected parent 110 retains its acquired feedback, memory timestamps, parameters, optimizer moments and random-stream ownership. Added computation 120 has private state and randomness. The new branch is not fed back into the old actor transition until the declared growth policy enables that coupling.

[0007] An additive readout may be $f_{\text{parent}} + g f_{\text{branch}}$ with a zero initial output gate. This gives initial output equality only when the branch does not perturb parent state or random draws. Frozen parent features permit a convex correction-head fit in a restricted embodiment. An output-preservation contract is not a guarantee that later learning improves prediction.

[0008] A hierarchy stores persistent memory at each level and produces upward views from queries or timer completions. A parent version contains references (ℓ, a, v) to lower-level memory address a and immutable version v, an issued time and a role or relative pose. A referenced version must be available before it is issued and issued before the parent version becomes available.

[0009] Instance identity, memory version and geometric role are separate fields. Distinct child addresses sharing a quadrant occupy distinct directory entries. A refreshed view can update a live child reference for a later parent publication without modifying an already published parent directory. A garbage collector retains transitive child references reachable from live parents, pending publications and in-flight messages.

[0010] The parent can query an earlier bound version after the child's current memory changes. Lazy transport evaluates that version at a query time not earlier than its availability. A local teacher can read the exact child version at its issued time, rather than substituting later child state. Observable shape or next-event targets are training outputs, not hidden identifiers supplied to inference.

[0011] FIG. 1 shows inputs 100, protected parent 110, private branch 120, binding and dispatch 130, output 140 and learning 150. FIG. 2 shows child publication 210, parent binding 220 and later versioned query 230. A digital circuit may implement directories in SRAM, timestamp comparisons in control logic and selected payload dispatch through

queues. Independent event schedules do not eliminate necessary local joins or buffering.

Hardware implementation

[0012] A digital implementation may use synthesizable register-transfer logic mapped to an FPGA or to an application-specific integrated circuit. Addressed state uses register banks or SRAM macros; arithmetic uses integer, fixed-point or floating-point datapaths. Ready/valid or request/acknowledge interfaces apply backpressure without accepting a message twice. Synchronous FPGA execution, locally clocked circuits and asynchronous circuits are alternative implementations, each preserving its declared causal ordering and numerical precision.

[0013] Before deployment, a reference trace can check selected addresses, values, timestamps, publication versions and output probabilities against the circuit at the declared numerical precision. Timing quantization, ties, cancellation, overflow and saturation are specified for the implementation. FPGA resource utilization, post-layout timing and measured energy are implementation-dependent quantities; no particular measured hardware result is required by this embodiment.

CLAIMS

1. A method of growing an event-driven learning system, comprising preserving an acquired parent processing path including its numerical state, timing state, optimizer ownership and random-stream ownership; adding a temporal branch with private state and randomness; initially combining a zero-output contribution of the branch with an acquired output; and training the added temporal computation while preventing undeclared branch feedback from changing the parent transition.

2. The method of claim 1, wherein parent parameters and features remain fixed while a correction readout is fitted.

3. The method of claim 1, further comprising publishing hierarchical parent-memory versions containing role-bearing references to immutable lower-memory versions.

4. The method of claim 3, wherein child identity is stored separately from geometric role so that different child memories sharing a role retain distinct references.

5. The method of claim 3, wherein a parent queries a bound lower-memory version after the lower memory has changed, using a query time not preceding availability of the bound version.

6. The method of claim 3, wherein local supervision reads a bound child version at its issued time.

7. A hierarchical processing apparatus comprising acquired-path storage, private branch storage, a versioned part directory and a growth controller configured to perform the method of claim 1.

8. A non-transitory computer-readable storage medium storing instructions that cause a

processor to perform the method of claim 1.

9. A method of hierarchical event processing, comprising publishing a parent-memory version with a directory of child-memory references, each reference identifying a child address, an immutable child-memory version, an issued time and a role; retaining distinct references for child identities sharing a geometric role; enforcing availability of a child version before its issued time and of the issued time before parent publication; and subsequently reading a referenced version after the child’s current memory changes.

10. The method of claim 9, further comprising training a local representation by reading the referenced child-memory version at the issued time for a parent-consumed view.

11. A hierarchical memory apparatus comprising immutable version storage, a role-bearing part directory and a timestamp-validated query controller configured to perform the method of claim 9.

ABSTRACT

A hierarchical event processor preserves an acquired actor path while adding temporal computation with private state, randomness and optimizer ownership. A branch initially contributes zero output without changing parent feedback. Published parent memories may retain role-bearing references to immutable lower-memory versions. Child identity is distinct from geometric role. Later queries can access bound versions after lower memories change, and local teaching can address the actual issued versions. Timestamp availability and live-reference retention govern access and collection.